1 - Langages Serveurs, JavaScript : côté serveur

Identifiez l'architecture d'un serveur web

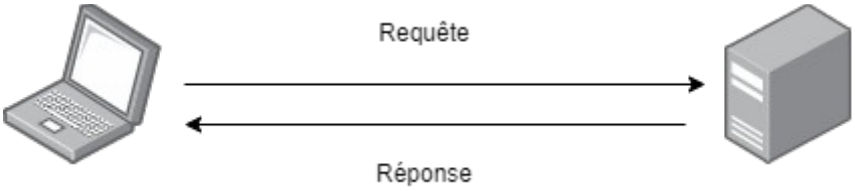
L'architecture d'un serveur web est la manière dont est conçu un serveur web, ainsi que la manière dont il est développé et déployé. Elle permet de définir les composants du serveur et leur disposition afin d'être en mesure d'offrir les opérations et services web requis.
Pour concevoir l'architecture interne d'un serveur, on prend en compte lors de sa conception la capacité physique du serveur et ses performances, on définit son système d'exploitation, les plateformes qu'il prend en charge, le type d'applications installées et sa connectivité (nombre d'utilisateurs connectés en simultané).
Mais le serveur a également une architecture logicielle qui définit la manière dont les tâches sont traitées. Généralement, un serveur web s'appuie sur l'architecture logicielle client-serveur.

Identifiez l'architecture client-serveur

L'architecture client-serveur est un mode de communication entre plusieurs composants d'un réseau. Chaque entité est considérée comme un client ou un serveur. Chaque logiciel de l'appareil client peut envoyer des requêtes à un serveur.
Cette architecture définit la manière dont sont réparties les tâches entre le serveur et l'utilisateur du service, le client. Le client démarre une session de communication avec le serveur qui écoute les requêtes entrantes. Il demande un service que lui fournit le serveur avec sa réponse. Le client peut être un ordinateur, une tablette, un smartphone ou une imprimante par exemple.
Il existe trois grandes catégories d'architecture : l'architecture deux tiers, l'architecture à trois niveaux et l'architecture à n-tiers.

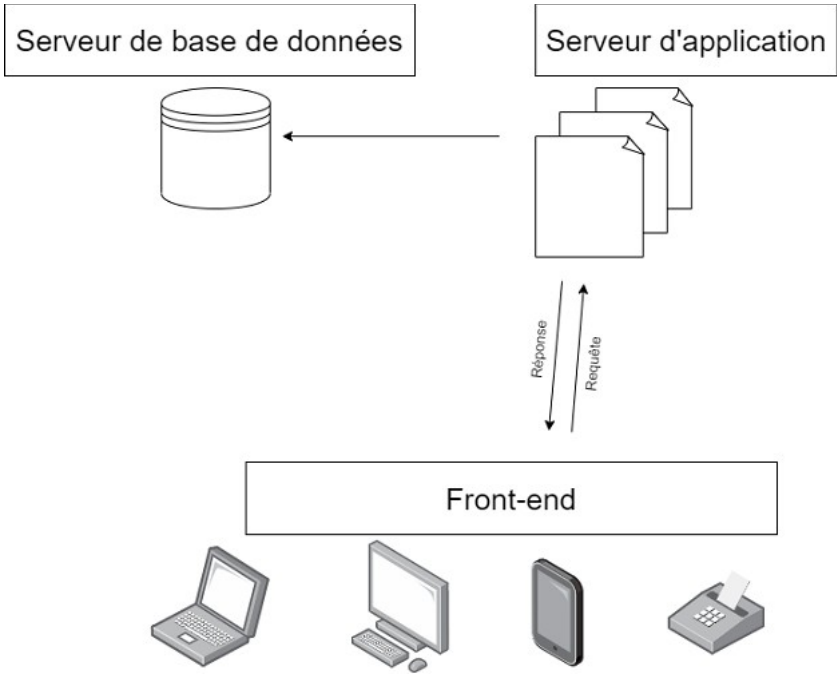
L'architecture deux tiers

Dans cette architecture très simple, on considère que toutes les ressources se trouvent sur le même serveur. On parle d'architecture à deux niveaux.



L'architecture à trois niveaux

Illustration architecture à trois niveaux



On peut constater sur ce schéma que l'architecture à trois niveaux se découpe en trois parties.

1. Le serveur d'application : C'est sur ce serveur que sont installés les logiciels permettant d'exprimer la logique de l'application. Il interroge la base de données avec laquelle il se connecte pour fournir une réponse à l'utilisateur.
2. Le serveur de base de données : Il contient les données nécessaires à l'application. C'est sur ce serveur que seront effectuées les opérations d'insertion, de suppression ou de modification de la donnée.
3. Le front-end : Il s'agit de la partie de l'application avec laquelle l'utilisateur peut interagir même s'il utilise une technologie ou une plateforme différente du serveur. C'est ici que l'on trouvera les modules de l'application qui permettent un affichage et des interactions avec tous les appareils existants.

En pratique : L'utilisateur envoi une requête au serveur en utilisant l'interface front-end. Par exemple en appliquant un filtre sur un site e-commerce permettant de n'afficher que les tee-shirts coûtants moins de 20 €. Le serveur d'application reçoit la requête et interroge la base de données en conséquence. Le serveur de base de données exécute la requête et envoie sa réponse au serveur d'application qui utilise le front-end pour afficher la liste des tee-shirts qui ont matchés avec la demande du client.

L'architecture à n-tiers

Dès qu'il y a plus de trois acteurs, on parlera d'architecture à n-tiers.

Utilisation de l'architecture client-serveur

Cette architecture est utilisée, par exemple, pour les sites e-commerce, pour les ERP mais aussi pour un groupe d'imprimantes ou une messagerie d'entreprise.
Avantages :

- Un contrôle de sécurité simplifié car tout est sur le même serveur.
- Les technologies supportant l'architecture client/serveur sont matures.
- Les utilisateurs utilisent simplement un client léger et tout le travail est supporté par le serveur.
- La rapidité pour retrouver et comparer de grandes quantités d'information.

Inconvénients :

- Défaut de service si trop de clients veulent communiquer sur le serveur simultanément.
- Si le serveur n'est plus disponible, plus aucun des clients ne fonctionne.
- Les coûts de mise en place et de maintenance sont élevés.

Je retiens : L'architecture standard

L'architecture client-serveur standard est composée de trois niveaux, le front-end, le serveur d'application et le serveur de base de données.
La machine cliente envoie une requête et le serveur, après traitement, lui envoie une réponse.
L'avantage de cette architecture est que les utilisateurs se servent simplement d'un client léger et tout le travail est supporté par le serveur.

Identifiez les couches de protocoles

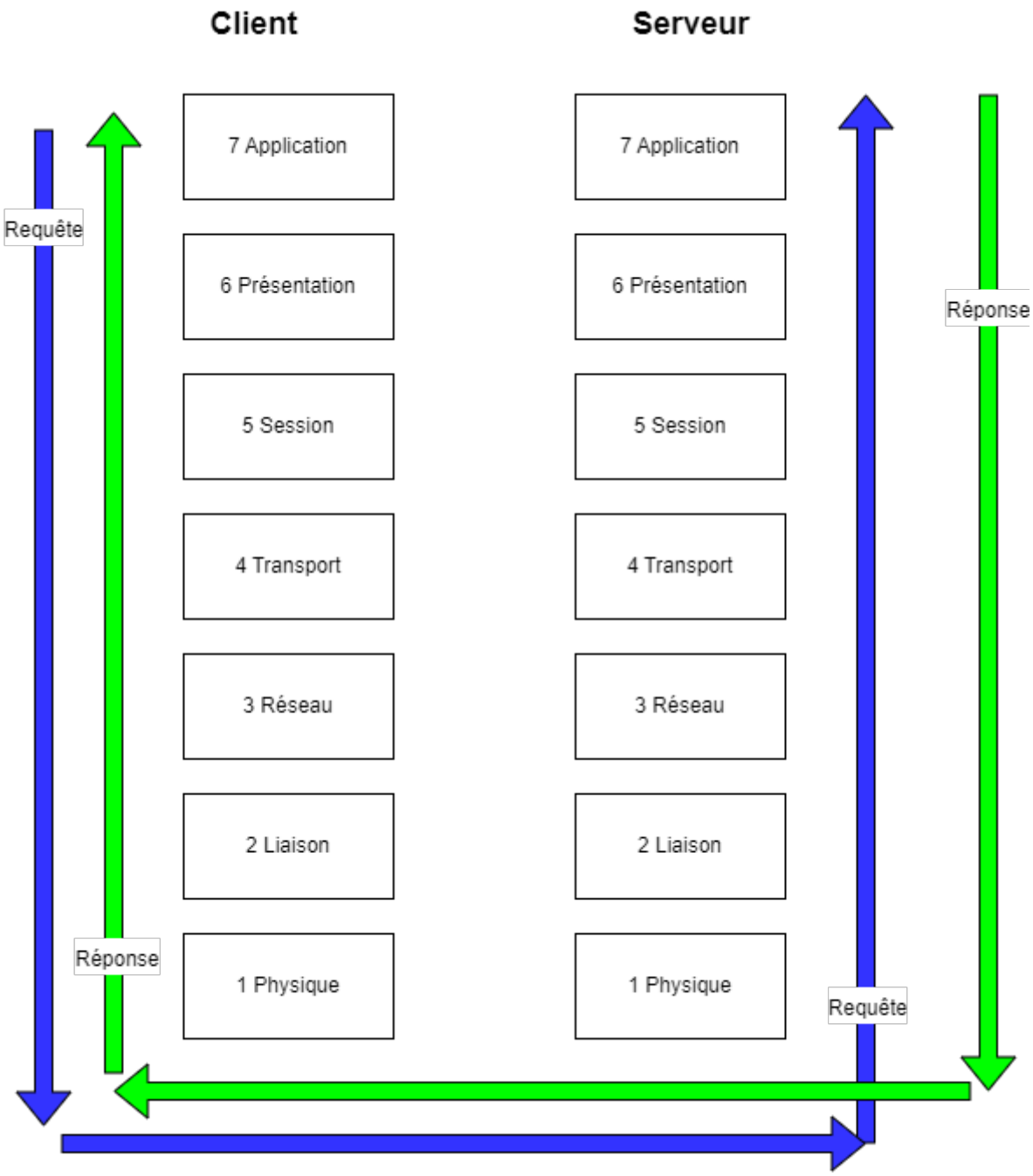
Un modèle de couche standard a été établi par l'International Standards Organisation (ISO), le modèle OSI. Mais ce modèle ne précise pas quels protocoles sont mis en jeu dans chaque couche et ne fait qu'évoquer le rôle de chacune d'entre elles.

Couches	Fonctions de la couche
APPLICATION	Initie ou accepte une requête réseau
PRESENTATION	Ajoute des informations de formatage, d'affichage, de cryptage
SESSION	Ajoute des informations de flux pour indiquer le départ d'un paquet
TRANSPORT	Ajoute des informations pour le traitement des erreurs des paquets
RESEAU	Ajoute un numéro d'ordre et des informations d'adressage au paquet
LIAISON	Ajoute des informations de contrôle d'erreurs d'un paquet
PHYSIQUE	Emet les paquets sur le réseau sous la forme d'un flot de bits bruts

Voici quelques protocoles utilisés dans chaque couche.

APPLICATION	FTP, SSH, SFTP, DNS, HTTP, IMAP, NFS, POP3, Telnet, FIX...
PRESENTATION	
SESSION	
TRANSPORT	TCP, UDP, RTP
RESEAU	IP, ICMP, IGMP, ARP
LIAISON	Ethernet, ATM, Token ring, SLIP
PHYSIQUE	électronique, radio, laser

Définissez le modèle OSI



Le plus connu des protocoles qui se base sur le modèle OSI est le modèle TCP/IP. C'est lui qui est employé sur le réseau internet.

Découvrez les protocoles de la pile TCP/IP

La pile TCP/IP comporte quatre couches : accès réseau, internet, transport et application.

Modèle OSI	Modèle TCP/IP
7 APPLICATION	APPLICATION
6 PRESENTATION	
5 SESSION	
4 TRANSPORT	TRANSPORT
3 RESEAU	INTERNET
2 LIAISON	ACCES RESEAU
1 PHYSIQUE	

La couche accès réseau

Cette couche décrit les caractéristiques physiques et le moyen utilisé pour acheminer les données sur la couche physique : câble Ethernet, Bluetooth, Wi-Fi.
Voici les principaux protocoles de cette couche :

- ICMP pour Internet Control Message Protocol, qui gère les erreurs et envoie des messages d'erreurs.
- IP pour Internet Protocol. C'est un des protocoles les plus importants car il permet l'adressage des paquets afin qu'ils arrivent à bonne destination.
- ARP pour Adress Resolution Control, qui fait correspondre les adresses logiques (IP) avec les adresses physiques (MAC).
- PPP pour Point to Point Protocol, qui permet d'établir une connexion distante par téléphone.

La couche internet

Le rôle de cette couche est d'injecter les paquets dans un réseau parfois dans le désordre. Le protocole transport se charge de remettre les paquets dans le bon ordre.

La couche transport

Les protocoles utilisés à ce niveau sont :

- TCP (Transmission Control Protocol), qui s'assure que les connexions entre deux ordinateurs sont établies et maintenues.
- UDP, qui est plus rapide mais moins fiable que TCP car il n'y a pas ou peu de vérifications sur la réception des paquets.

On utilisera UDP pour le streaming ou la vidéo conférence ou la perte de données est moins importante que la rapidité. Pour tout le reste, ce sera TCP.

La couche application

Cette couche regroupe tous les protocoles dit de haut niveau comme :

- FTP pour File Transfer Protocol, qui est utilisé pour envoyer des fichiers d'un serveur vers un client.

- SMTP pour Simple Mail Transfer Protocol, qui se charge de la transmission des courriers électroniques.
- HTTP pour Hypertext Transfer Protocol, dont les clients les plus connus sont les navigateurs web.
- DNS pour Domain Name System, qui est le répertoire d'adresse sur internet. Le DNS traduit les noms de domaine en adresses IP afin que les navigateurs puissent charger les ressources web.
- TELNET, qui permet d'établir une connexion à un hôte distant et de gérer les données locales.
- BGP/EGP pour Border Gateway Protocol / Exterior Gateway Protocol, qui est utilisé pour gérer la transmission des données entre les réseaux.
- SNMP pour Simple Network Management Protocol, qui permet la gestion des équipements sur un réseau.
- POP 3 (Post Office Protocol version 3) et IMAP 4 (Internet Message Advertising Protocol version 4), qui permettent de se connecter à un serveur de messagerie et de récupérer son courrier électronique.

C'est dans la couche application qu'est choisi un protocole de transport adapté au service demandé.

Note : Fetch est la méthode la plus utilisé pour discuter avec du back-end depuis du JS front-end.

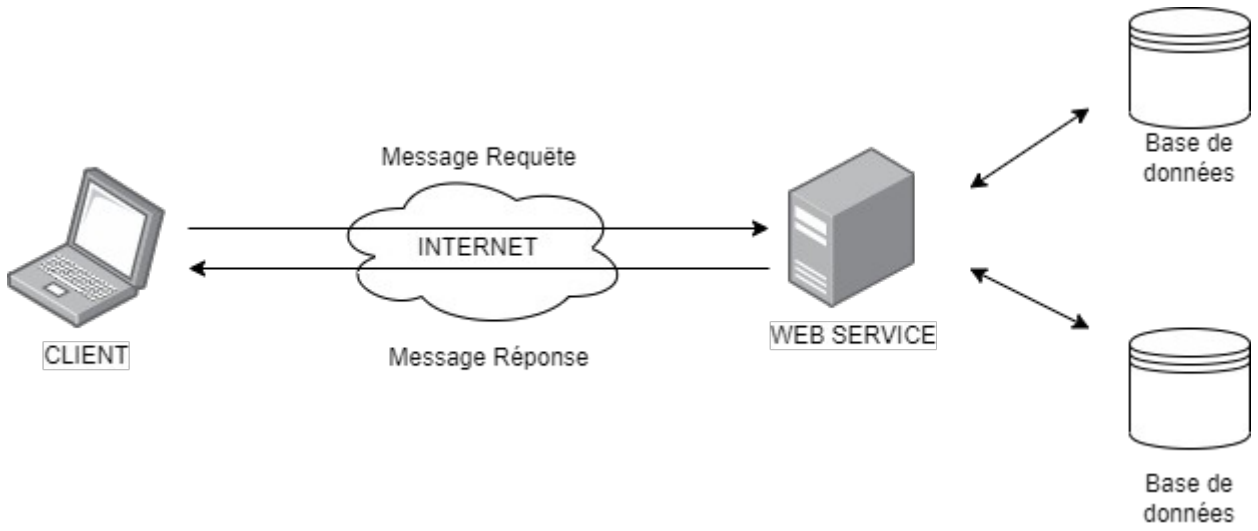
Définissez le web service

Généralités

Un web service est un logiciel standardisé permettant la communication entre les applications clients et serveur sur le web. Les applications web modernes sont développées à l'aide de différents langages de programmation comme Java, C# avec son framework .Net, JavaScript avec des frameworks comme Angular JS ou React JS et l'environnement d'exécution Node.js. Aussi, il faut s'assurer que ces diverses applications puissent communiquer entre elles. Tel est le rôle d'un web service. Les plus connus sont SOAP pour Simple Object Access Protocol, REST pour Representational State Transfer ou HTTP pour Hypertext Transfer Protocol.
L'ensemble des services d'internet accessibles *via* le web forme ce que l'on appelle le cloud. Si on devait donner une définition du cloud, ce serait un « nuage » de services.
Les interfaces des web services sont décrites en détail par des documents WSDL (Web Service Description Language). Cette description détaillée permet à un utilisateur de créer une application cliente capable de converser avec elles. On y trouve les méthodes pouvant être utilisées, leurs signatures et les points d'accès du service comme le port ou l'URL.

Fonctionnement d'un web service

Le client effectue une série d'appels au serveur sur lequel est hébergé le web service. Pour cela, il envoie des requêtes en utilisant des appels de procédure distante ou Remote Procedure Calls. Un service web est composé principalement des données transférées entre le client et le serveur. Ces données sont généralement agencées en XML. Pour envoyer les données XML d'une application à une autre, les services web utilisent souvent le SOAP ou la requête et les réponses sont des messages XML transportés via HTTP.



Je retiens : Le protocole réseau

Un protocole réseau un ensemble de règles utilisé pour échanger l'information sur un réseau.
Le modèle utilisé pour communiquer sur le réseau internet s'appelle TCP/IP.
Il est formé de quatre couches de protocoles : accès réseau, internet, transport et application.
Chaque couche possède des protocoles avec des fonctions précises.

Internet est un réseau permettant de fournir des services et le World Wide Web est l'un de ces services.
On utilise les interfaces web pour accéder aux services d'internet comme la messagerie électronique (Gmail, Yahoo) ou le transfert de fichiers (Dropbox, WeTransfer).
Un web service est un logiciel standardisé permettant la communication entre des applications clients et serveur écrites dans différents langages. Les données sont envoyées en XML d'une application à une autre en utilisant SOAP ou la requête et les réponses sont des messages XML transportés via HTTP.

Découvrez le moteur Google V8

V8 est un moteur JavaScript open source écrit en C++ et développé par le projet Chromium. Il a été créé par Lars Bak au centre de développement de Google en Allemagne. La première version du moteur V8 est publiée le 2 septembre 2008. Il est conçu au départ pour améliorer les performances d'exécution de JavaScript dans les navigateurs.

Principe du moteur V8

Pour augmenter la vitesse d'exécution, le moteur V8 traduit le code JavaScript en langage machine (assembleur) grâce à un compilateur JIT (Just-In-Time). Il n'utilise pas d'interpréteur.

Différence avec les autres moteurs modernes

Contrairement aux moteurs Rhino de la fondation Mozilla ou JavaScriptCore d'Apple, il ne produit pas de code ou de bytecode intermédiaire.

Utilisation du moteur V8

V8 est utilisé à la fois dans les navigateurs ou intégré dans des projets comme moteur performant. Par exemple, il est utilisé dans le navigateur Google Chrome ou dans la structure des projets Electron.js.

Le sandboxing : la fonctionnalité-clé du moteur V8

Une sandbox - ou bac à sable - est un environnement pour exécuter des logiciels qui est isolé des autres environnements, ce qui permet de faire des tests isolés dans un système.
Le sandboxing a pour avantage de protéger le système d'exploitation de la machine. Cet environnement garantit que les fonctions JavaScript s'exécutent séparément. L'exécution d'une partie du code n'affecte aucune autre partie du code. Contrairement au lancement d'exécutables sur machine virtuelle, le sandboxing du moteur V8 n'affecte en rien les performances.

Identifier les caractéristiques du V8

V8 se retrouve côté serveur dans des technologies telles que les bases de données MongoDB ou Couchbase. Parce qu'il permet d'exécuter du code JavaScript en dehors du navigateur et qu'il est performant, il fait partie de la combinaison déployée pour concevoir l'environnement d'exécution Node.js et ainsi être utilisé côté serveur. Node.js n'a pas été conçu pour fonctionner avec un autre moteur Javascript et, s'il n'en avait plus, il perdrait sa capacité d'exécution du langage. En réalité, tous les modules natifs fournis avec

node.js reposent sur l'interface V8 spécifique entre C ++ et Javascript. La maintenance et l'évolution du projet Node.js est donc tributaire de l'évolution du moteur V8.

Je retiens : Le fonctionnement du moteur V8

V8 est un moteur qui permet d'exécuter du javascript en dehors du navigateur.
On le retrouve côté serveur dans les technologies MongoDB, Couchbase et Node.js.
Son système de sandboxing permet de créer un environnement de test isolé.
Node.js ne peut pas fonctionner sans V8.

Définissez le principe de programmation évènementielle

La programmation évènementielle est un paradigme de programmation qui s'appuie sur les évènements. Le programme définit la manière dont il réagit aux différents évènements comme un clic sur un bouton de souris, une entrée au clavier ou le changement d'état d'une variable, par exemple.
C'est également une technique d'architecture logicielle dans laquelle l'application possède une boucle principale composée de deux sections : la détection des évènements et la gestion des évènements. C'est ce paradigme qui est utilisé dans Node.js notamment, mais aussi pour Android ou Visual Basic. On retrouve la programmation évènementielle dans la programmation d'automates ou le développement d'interfaces graphiques.

Principe

Le principe est simple. Lorsqu'un événement survient dans l'environnement, l'exécution d'actions est déclenchée automatiquement. On note ces associations évènement->action. L'ordre d'exécution des instructions n'est pas prévu à l'avance. C'est l'utilisateur qui contrôle le programme.

Les évènements

Un évènement est un message envoyé à l'application. Cet évènement peut avoir plusieurs origines :

- Une action de l'utilisateur (clic ou survol sur un élément de l'interface graphique, appui sur une touche du clavier, etc.) ;
- Un évènement système (chargement d'un fichier, déclenchement d'une temporisation, etc.).

Utilisez la programmation événementielle

La programmation événementielle s'oppose à la programmation séquentielle.
Le fait de pouvoir réagir aux évènements s'avère particulièrement adapté lorsqu'il s'agit

de gérer de nombreuses interactions avec l'utilisateur. C'est pourquoi on l'emploie pour réaliser des interfaces graphiques homme-machine (GUI, Graphical User Interface).
Le web fourmille de ce type d'interfaces sur lesquelles on cherche à donner le contrôle à l'utilisateur. Les développeurs web utilisent donc sans cesse ce paradigme pour développer des interfaces graphiques dynamiques toujours plus riches en fonctionnalités.

A

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <button id="submitBtn">Confirmer</button>
  <script>
    window.onload = function() {
      console.log('Page loaded');

      let submitBtn = document.querySelector('#submitBtn');
      console.log(submitBtn);

      submitBtn.addEventListener('click', () => {
        console.log('Button clicked')
      })
    };
  </script>
</body>
</html>
```

note : ici on voit comment sélectionner un bouton du HTML avec JS pour ensuite déclencher une action après un évènement let que le clic du bouton par exemple.

Je retiens : La programmation événementielle

La programmation évènementielle est un paradigme de programmation fondé sur les évènements.
Lorsqu'un événement survient, l'exécution d'actions est déclenchée automatiquement.
Un évènement est une action de l'utilisateur ou un changement dans le système.
On l'utilise pour développer des interfaces graphiques

Rappels : notion de variable et de type

Un algorithme est la description des opérations nécessaires pour arriver à un résultat à partir de valeurs d'entrée. Un algorithme manipule des objets sur lesquels et avec lesquels il peut effectuer des actions. On appelle ces objets des variables. Pour manipuler les variables, on effectue des opérations.
Une variable peut posséder un des quatre types de base :

- Entier noté Ent
- Réel noté Réel
- Booléen noté Bool
- Caractère noté Car

La manière de manipuler des variables la plus courante est le calcul. Pour cela, on utilise des opérateurs.

Liste des opérateurs

Opérateur	Notation	Type des variables	Type du résultat
+ et – (unaires)	+ -	Entier ou Réel	Celui des variables
Négation	Non	Booléen	Booléen
Multiplication	*	Entier ou Réel	Entier ou Réel
Division de réels	/	Réel	Réel
Quotient d'entiers	Div	Entier	Entier
Modulo (reste de la division)	Mod	Entier	Entier
Addition	+	Entier ou Réel	Entier ou Réel
Soustraction	-	Entier ou Réel	Entier ou Réel
Comparaison	>, <, >=, <=, =, <>	Tous	Booléen
Puissance	**	Entier ou Réel	Entier ou Réel
Et	Et	Booléen	Booléen
Ou	Ou	Booléen	Booléen

Exemple d'algorithme simple

Un problème se décompose en sous-problèmes résolus dans un certain ordre. Chaque sous-problème produit un résultat qu'on doit pouvoir utiliser dans un autre sous-problème. C'est l'objectif des variables et il est donc très important de bien les nommer.
Le problème : écrivez un algorithme qui demandera la saisie du nombre de jours de congé pris et affichera le nombre de jours de congé qui reste à prendre.
Un algorithme commence toujours par la borne Début et se termine toujours par la borne Fin. Vous utiliserez Ecrire () qui permet d'afficher et Lire qui permet de capturer une valeur d'entrée dans le programme et de l'affecter avec le signe <- à une variable.
Nous verrons également comment écrire des commentaires avec Co et Fco.

```
Début
Co Ce programme calcule le nombre de jours de congé qu'il reste à prendre à partir du nombre de
jours déjà pris. Vous avez besoin de trois variables : la première pour ranger le résultat du calcul, la
seconde pour ranger la saisie de l'utilisateur et la troisième pour représenter le nombre total de jours
de congé Fco
Ent CONGEPRIS, CONGERESTANT = 21
Ent CONGETOTAL = 21
Co On informe l'utilisateur qu'il va devoir saisir une valeur.  Fco
Ecrire ("Saisissez le nombre de jours de congé que vous avez pris")
Co Ci-dessous le programme attendra que l'utilisateur entre une valeur pour CONGEPRIS.  Fco
CONGEPRIS <- LIRE
Co On effectue le calcul pour obtenir le reste de jours de congé.  Fco
CONGERESTANT = CONGETOTAL – CONGEPRIS
Co On affiche le résultat en concaténant des chaînes de caractères avec la variable.  Fco
Ecrire ("Il vous reste ", CONGERESTANT, "jours de congé.")
Fin
```

Les commentaires sont des informations utiles pour comprendre le programme mais ne sont exécutés.

Utilisez une variable

Les conditions

Dans la plupart des problèmes, certaines actions ne sont à effectuer que si certaines conditions sont remplies. Ces choix sont conditionnés par un booléen qui exécute le traitement du bloc si sa valeur est vraie. On peut ainsi améliorer l'algorithme après la saisie de l'utilisateur, en écrivant plutôt :

```
Si CONGETOTAL < CONGEPRIS
Alors
    Ecrire ("Vous n'avez pas assez de jours de congé.")
Sinon
    CONGERRESTANT = CONGETOTAL – CONGEPRIS
    Ecrire ("Il vous reste ", CONGERESTANT, " jours de congé.")
Finsi
```

Les structures itératives

Dans la plupart des problèmes, il faut exécuter plusieurs fois certaines actions. Si le nombre de répétitions est trop grand, il devient compliqué de réécrire plusieurs fois la même séquence de code. Pour cela, vous disposez de quatre types d'itérations qui

permettent de résoudre cette problématique.

La boucle « Tant que »

Tantque condition **Faire**

Actions

Refaire

La boucle « Faire … Tant que » ou « Tant que avec le test en fin d'itération »

Faire

Actions

Tantque condition **Refaire**

La boucle « Jusqu'à »

Jqa condition **Faire**

Actions

Refaire

La boucle « Faire … Jusqu'à » ou « Jusqu'à avec le test en fin d'itération »

Faire

Actions

Jqa condition **Refaire**

Condition est un booléen de résultat Vrai ou Faux. Actions est un bloc de code de traitement.
Pour les itérations dont les tests se font en fin de boucle, les actions sont effectuées au moins une fois.

Les tableaux

Un tableau est un ensemble de valeur du même type. Il peut être à une ou plusieurs dimensions.
Exemple de notation pour un tableau qui regroupe les mois de l'année : Car MOIS (1, 12)
On précise le type, l'identificateur et, entre les parenthèses, les bornes inférieures et supérieures.
Pour savoir combien d'éléments contient le tableau, on soustrait la borne inférieure + 1 à

la borne supérieure. On a 12 -1 +1 soit 12 éléments.
Le tableau peut être à plusieurs dimensions et, dans ce cas, on note :
Car MOIS (1, 12, 1, 12, 1, 12, … ,1, 12).

Isolez une partie de traitement

Procédure :

Procédure NOM_PROCEDURE (**Type-1** Info-1, ..., **Type n** Info-n)

Traitement de la procédure / Corps de la procédure

Fin_Procédure

Fonction :

Fonction NOM_FONCTION (**Type-1** Info-1, ..., **Type n** Info-n)

Type_résultat

Corps de la fonction contenant une expression **Retourne** un résultat au programme

Fin_Fonction

Ces mécanismes permettent de convertir les sous-problèmes en procédures ou fonctions et de traiter la problématique de manière générale grâce aux paramètres. On utilisera ces mécanismes autant de fois que nécessaire dans le programme en utilisant son nom et en lui passant les bons paramètres.

Je retiens : L'algorithmie

Un algorithme est une recette écrite avec sa propre syntaxe.
Grâce aux variables, l'algorithme peut manipuler des valeurs.
Les traitements conditionnés permettent de poser une condition à l'exécution d'un bloc de code.
Les itérations permettent d'exécuter des actions récurrentes.
Les procédures ou les fonctions permettent d'isoler une partie de traitement et de les réutiliser partout dans le programme.